Oficina Mecânica — Fase 4

Microsserviços, saga orquestrada, banco por serviço e CI/CD independente.

Danilo Evangelista Canato — RM 372411
Guilherme Fumagali Marques — RM 372421

PosTech FIAP · Arquitetura de Software

Ponto de partida

Monólito em Clean Architecture, um banco PostgreSQL, EKS, API Gateway e New Relic.

FASE 3

O que existia

Um serviço e um banco.

Orçamento calculado a partir dos itens da OS, aprovação por token enviado por e-mail.

Sem pagamento, sem mensageria e sem transação explícita: a reserva de peça gravava estoque e OS em commits separados.

AVALIAÇÕES

Pontos registrados

F1: Domain Storytelling não localizado.

F1: contêiner como root.

F3: login só por CPF, sem segundo fator.

F3: produção configurada e não provisionada.

FASE 4

Tratamento

Domain Storytelling do fluxo distribuído.

Imagens não-root e securityContext restritivo.

Código de uso único por e-mail.

Produção mantida configurada, sem provisionar, por custo.

Três microsserviços

Decomposição por subdomínio, a partir dos contextos do Event Storming; migração em Strangler Fig.

TECH-CHALLENGE-1

OS Service

Cliente, veículo, catálogo e estoque.

Abertura da OS, status e histórico.

Orquestrador da saga.

PostgreSQL próprio.

OFICINA-BILLING-SERVICE

Billing Service

Orçamento e envio para aprovação.

Decisão do cliente.

Pagamento no Mercado Pago e estorno.

PostgreSQL próprio.

OFICINA-EXECUTION-SERVICE

Execution Service

Fila de execução.

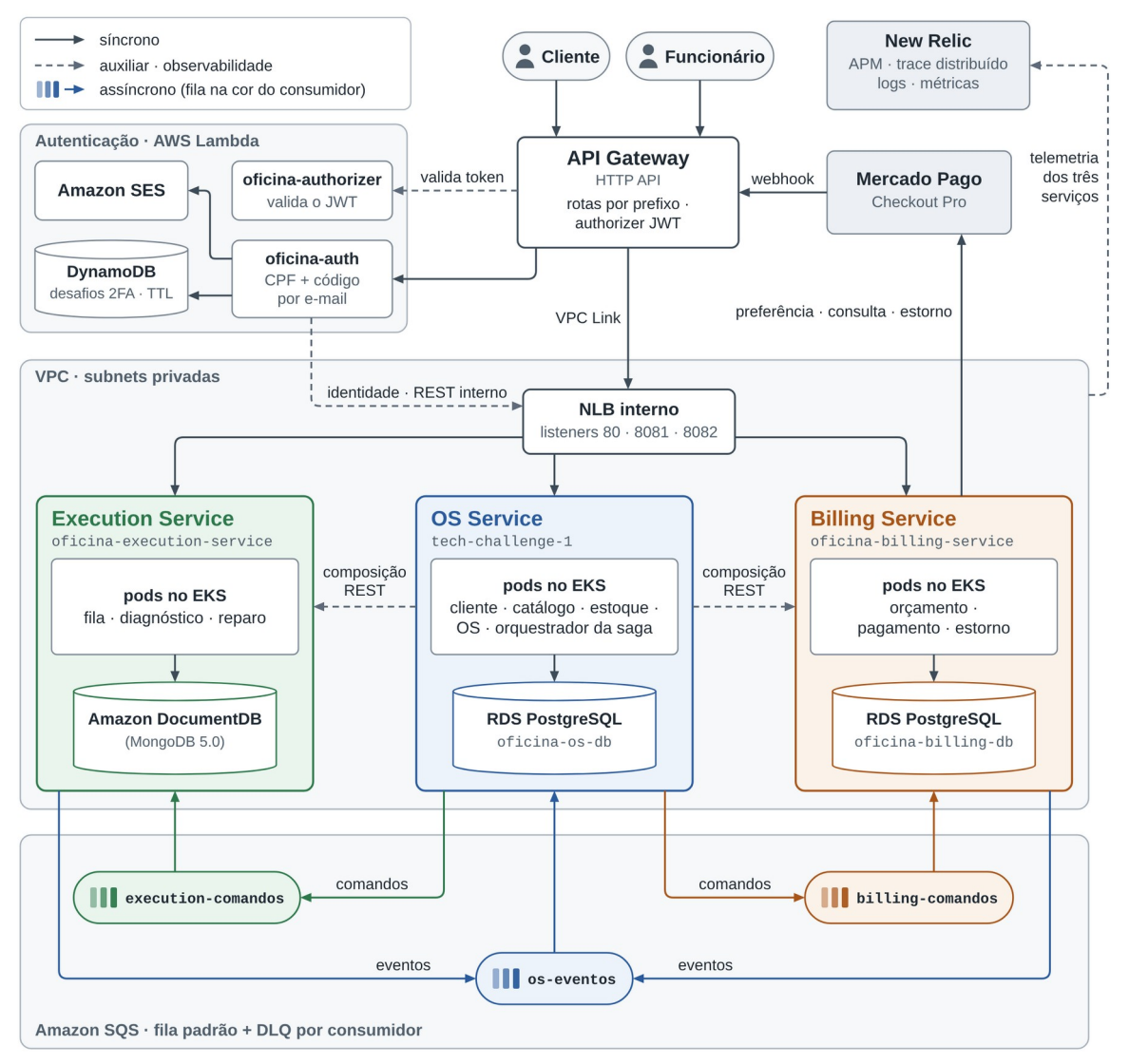
Diagnóstico, reparo e apontamentos.

Falha e finalização.

MongoDB (DocumentDB) próprio.

Catálogo e estoque ficaram no OS: a reserva de peça e o estorno continuam sendo transações locais.

Arquitetura final



Comunicação entre os serviços

Síncrona onde a resposta precisa ser imediata; assíncrona nos passos da saga.

REST via API Gateway	Cliente e funcionário; rotas por prefixo para cada serviço
REST interno	Composição do acompanhamento da OS (Resilience4j) e identidade para o login
SQS · execution-comandos	IniciarDiagnostico, ReabrirDiagnostico, IniciarReparo, CancelarExecucao
SQS · billing-comandos	GerarOrcamento, CancelarOrcamento, EstornarPagamento
SQS · os-eventos	Respostas e eventos de Billing e Execution para o orquestrador
Webhook	Mercado Pago → Billing, com assinatura HMAC e consulta do pagamento na API

Fila padrão com DLQ (5 tentativas); envelope comum com mensagemId, sagaId e osId; contrato publicado em AsyncAPI.

Saga: por que orquestração

Recomendação da aula 2: coreografia para sagas simples, orquestração nas demais.

COREOGRAFIA

Descartada

Fluxo espalhado entre os serviços.

Cada participante precisaria conhecer a ordem das compensações dos outros.

Risco de dependências cíclicas.

CARACTERÍSTICAS DA SAGA

Não é simples

Sete etapas.

Duas esperas humanas: aprovação e pagamento.

Três caminhos de falha, com compensações em ordens diferentes.

Passo pivô: pagamento aprovado.

ORQUESTRAÇÃO

Adotada

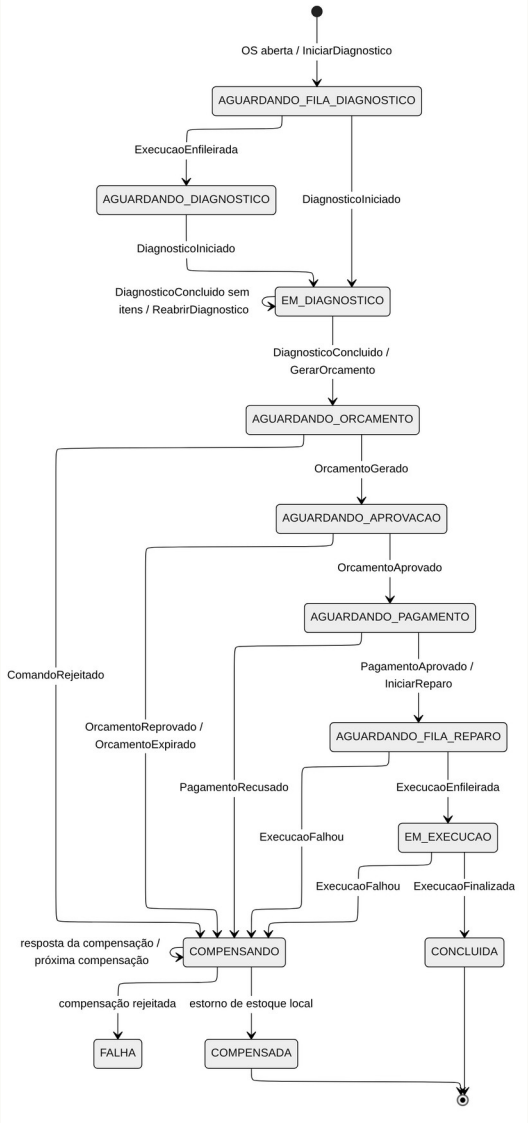
Orquestrador no OS Service, dono da OS.

Estado persistido e consultável por OS.

Lógica em uma classe de domínio testada linha a linha.

Billing e Execution só respondem a comandos.

Estados da saga



Compensações e falta de isolamento

Pessimistic view: primeiro o que impede trabalho novo, por último o que devolve recurso.

Reprovação do orçamento	cancelar execução → estornar estoque → OS CANCELADA
Pagamento recusado	cancelar orçamento → cancelar execução → estornar estoque
Falha na execução	estornar pagamento no Mercado Pago → cancelar orçamento → estornar estoque
Diagnóstico sem itens	reabrir diagnóstico
Serviço indisponível	mensagem aguarda na fila; reenvio após o prazo, até 3 vezes

Reread value: o orquestrador relê OS e saga com trava otimista (@Version) antes de aplicar cada evento.

Garantias de entrega

Gravar no banco e publicar na fila são operações em sistemas diferentes.

TRANSAÇÃO LOCAL

Unidade de trabalho

Todo caso de uso que escreve roda em uma transação explícita.

Corrigiu a reserva de peças herdada do monólito.

ENVIO

Transactional Outbox

Mensagem gravada na mesma transação da mudança de estado.

Relay a cada 500 ms com FOR UPDATE SKIP LOCKED.

CONSUMO

Inbox idempotente

mensagemId gravado na transação do efeito.

Reentrega descartada; comando repetido responde de novo sem repetir o efeito.

FALHA

Prazo e DLQ

Comando sem resposta é reenviado.

Cinco falhas de processamento: DLQ e alerta crítico.

Banco de dados por serviço

Persistência poliglota: cada banco escolhido pelo formato dos dados e pela consistência necessária.

OS SERVICE

PostgreSQL

Entidades relacionadas e invariantes em constraint.

Reserva de peça e item da OS na mesma transação.

BILLING SERVICE

PostgreSQL próprio

Unicidade de pagamento e de notificação.

Conciliação por status e data.

Payload do gateway em JSONB.

EXECUTION SERVICE

MongoDB · DocumentDB

Agregado lido e gravado inteiro, com listas que variam por OS (aula 2 de Data Engineering).

Transações para o outbox.

AUTENTICAÇÃO

DynamoDB

Desafio do segundo fator: acesso por chave e expiração por TTL, sem servidor ligado.

Descartados para a execução: Redis (cache), Cassandra (operação), Neo4j (relacionamento não é o centro), DynamoDB (sem modelagem nas aulas).

Integração com o Mercado Pago

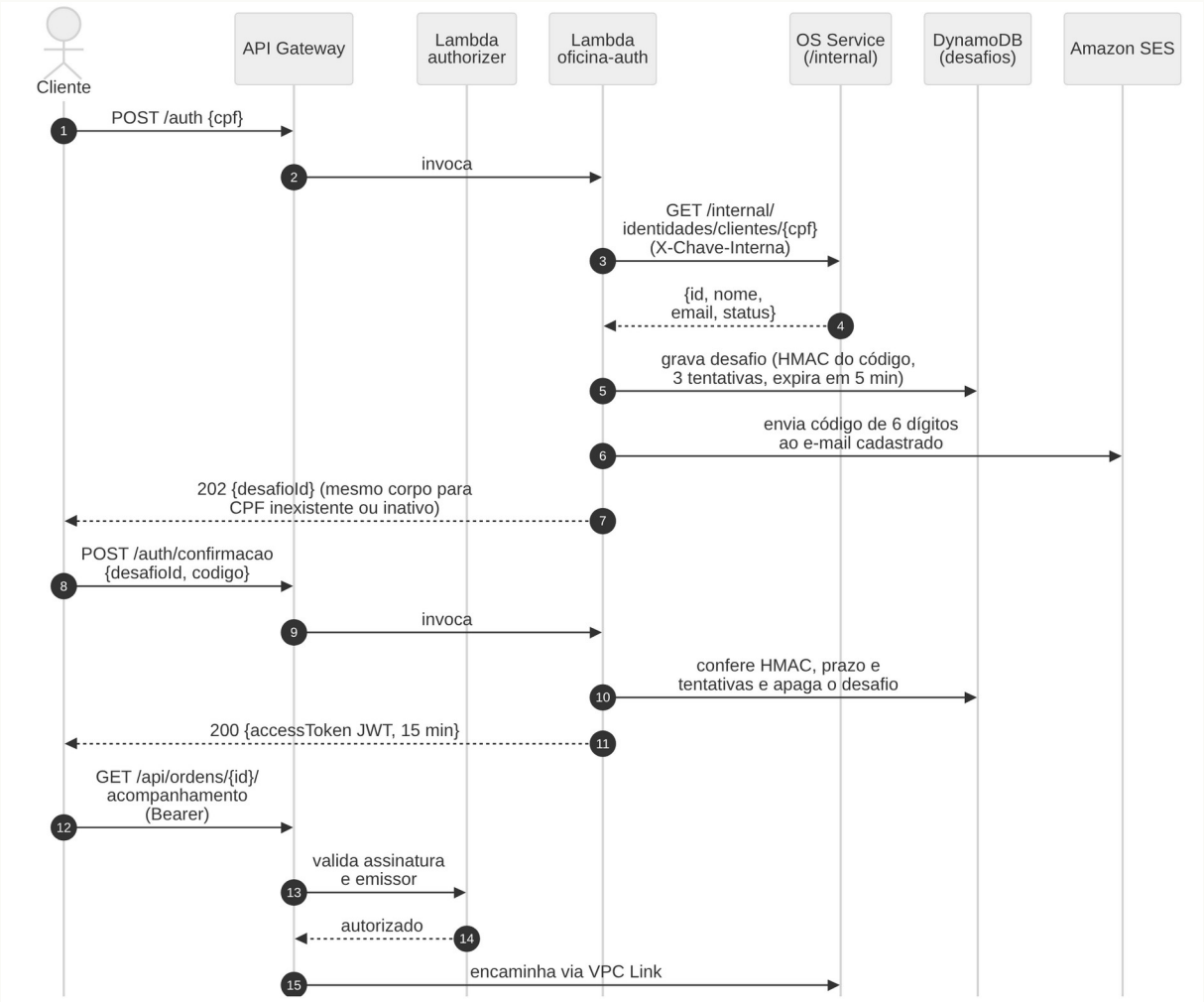
Checkout Pro pela API REST; o resultado sempre consultado no gateway.

Cobrança	POST /checkout/preferences · modo binário · referência = orçamento · idempotência
Notificação	Webhook com x-signature (HMAC-SHA256) validada
Consulta	GET /v1/payments/{id} com o token do vendedor
Reconciliação	Job a cada 60 s: notificação perdida ou credencial de teste sem webhook
Estorno	POST /v1/payments/{id}/refunds como compensação do passo pivô

Cartões de teste: titular APRO aprova; OTHE recusa e aciona a compensação. Gateway simulado nos testes e no ambiente local.

Autenticação com segundo fator

Encerra o débito da Fase 3: o CPF deixa de bastar para obter o token.



Testes, qualidade e entrega

Pipeline independente por serviço; main e develop protegidas por PR e checks.

TESTES

Pirâmide por serviço

Unitários de domínio e casos de uso.

Integração com Testcontainers: PostgreSQL, MongoDB e LocalStack.

Contrato OpenAPI versionado.

BDD

Saga em Gherkin

Cucumber em português no OS Service.

Fluxo completo, reprovação, recusa, falha com estorno, duplicata e prazo vencido.

QUALIDADE

Cobertura e Sonar

JaCoCo ≥ 80% interrompe o build.

SonarCloud com quality gate em cada PR.

ENTREGA

Deploy no EKS

Imagem no ECR por commit.

kubectl apply -k no overlay do ambiente, rollout e teste de fumaça.

Infra do serviço em Terraform no próprio repositório.

Rastreamento dos fluxos distribuídos

New Relic com uma aplicação por serviço e o mesmo trace atravessando a fila.

Trace pela fila	contexto gravado no outbox e propagado nos atributos da mensagem SQS
Correlação	os.id e saga.id em toda transação e nos logs estruturados
Dashboard Saga	iniciadas, concluídas, compensadas por motivo, duração por estado, fila
Alertas	mensagem na DLQ, saga em FALHA, pico de compensações, falha no Mercado Pago
Runbooks	cada alerta aponta para o procedimento versionado no repositório

DEMONSTRAÇÃO

Ambiente de homologação

1. Login em duas etapas e abertura da OS.
2. Diagnóstico, orçamento por e-mail, aprovação e pagamento aprovado no Mercado Pago.
3. Reparo concluído: OS finalizada e acompanhamento consolidado.
4. Pagamento recusado: compensações e OS cancelada com as peças devolvidas.
5. Falha na execução: estorno no Mercado Pago.
6. Billing fora do ar: mensagens aguardam na fila e a saga retoma.
7. PR → testes, Sonar e deploy de um serviço; trace e dashboard no New Relic.

Obrigado.

github.com/Guilherme-Fumagali/tech-challenge-1

github.com/Guilherme-Fumagali/oficina-billing-service

github.com/Guilherme-Fumagali/oficina-execution-service

github.com/Guilherme-Fumagali/oficina-auth-lambda

github.com/Guilherme-Fumagali/oficina-infra-k8s

Danilo Evangelista Canato — RM 372411 · Guilherme Fumagali Marques — RM 372421